UNIVERZITET U NIŠU
ELEKTRONSKI
FAKULTET

High Availability rešenja u MongoDB bazi podataka

Seminarski rad

Studijski program: Računarstvo i informatika
Modul: Softversko inženjerstvo

Student:
Anastasija Savov
Broj indeksa: 1924

Mentor:
Prof. dr Aleksandar Stanimirović

Niš, 2026.

Sadržaj

1 Uvod **3**

2 Osnove visoke dostupnosti u distribuiranim sistemima **4**
2.1 Definicija i koncepti 4
2.2 CAP teorema i njegova primena u MongoDB-u 4
2.3 Replikacija kao osnova visoke dostupnosti 4

3 Replica setovi **5**
3.1 Arhitektura replica set-a 5
3.2 Oplog (Operations Log) 6
3.3 Inicijalna sinhronizacija 6
3.4 Konfiguracija replica set-a 7
3.5 Parametri 8

4 Automatski failover i izbor primarnog čvora **9**
4.1 Mehanizam detekcije kvara 9
4.2 Protokol izbora (Election Protocol) 10
4.3 Vreme failover-a 10
4.4 Read i Write Concern u kontekstu visoke dostupnosti 12
4.4.1 Write Concern 12
4.4.2 Read Concern 13
4.4.3 Read Preference 13

5 Shardovani klasteri **14**
5.1 Arhitektura shardovanih klastera 14
5.2 Shard ključ i distribucija podataka 14
5.3 Balansiranje podataka 17
5.4 Targeted vs. Broadcast operacije 18

6 Backup i strategije oporavka **19**
6.1 Značaj backup-a u HA okruženju 19
6.2 mongodump i mongorestore 19
6.3 Point-in-Time Recovery 20
6.4 Odloženi sekundarni čvorovi kao zaštita od grešaka 21

7 Monitoring i dijagnostika **22**
7.1 Značaj monitoringa za visoku dostupnost 22
7.2 serverStatus i mongostat 22
7.3 Praćenje replica set stanja 22
7.4 Preporučene metrike za alerting 23

8 Geografski distribuirani deployment **24**
8.1 Multi-data-center replica set 24
8.2 Zonal tagging i Read Preference 24

9 WiredTiger storage engine i journaling **25**
9.1 Uloga storage engine-a u HA 25
9.2 Journaling 25

10 Zaključak **26**

11 Literatura **27**

1 Uvod

Visoka dostupnost (High Availability, HA) predstavlja sposobnost sistema da nastavi sa radom bez prekida ili sa minimalnim prekidima čak i u slučaju kvarova pojedinih komponenti. U savremenim distribuiranim sistemima, gde se podaci čuvaju i obrađuju na velikom broju čvorova, osiguranje kontinuiteta rada postaje jedan od ključnih zahteva. Baze podataka, kao centralne komponente informacionih sistema, moraju biti projektovane sa posebnim akcentom na dostupnost, jer svaki prekid u radu direktno utiče na korisnike i poslovne procese [1].

MongoDB, kao vodeći dokumentni sistem za upravljanje bazama podataka, pruža sveobuhvatan skup mehanizama za postizanje visoke dostupnosti. Za razliku od tradicionalnih relacionih baza podataka koje su primarno projektovane za rad na jednom serveru, MongoDB je od samog početka projektovan za distribuirano okruženje [2]. Ovo se odražava u osnovnim arhitekturalnim konceptima sistema - replica sets obezbeđuju automatsku replikaciju, dok shardovani klasteri pružaju horizontalnu skalabilnost uz očuvanje dostupnosti.

U kontekstu visoke dostupnosti, ključne metrike su:

- **RTO** (Recovery Time Objective) - maksimalno prihvatljivo vreme oporavka sistema nakon kvara
- **RPO** (Recovery Point Objective) - maksimalno prihvatljivi gubitak podataka izražen kroz vremenski period
- **MTTR** (Mean Time To Recovery) - prosečno vreme potrebno za oporavak sistema
- **MTBF** (Mean Time Between Failures) - prosečno vreme između kvarova

Cilj dobro projektovanog HA sistema je da minimizuje RTO i RPO uz prihvatljive troškove infrastrukture. MongoDB postiže ove ciljeve kombinacijom automatske replikacije podataka, automatskog preuzimanja uloge (automatic failover) i fleksibilnih mehanizama za konzistentnost čitanja i pisanja [2].

Ovaj seminarski rad analizira ključne mehanizme visoke dostupnosti u MongoDB-u: arhitekturu replica set-ova i proces izbora primarnog čvora, shardovani klasteri kao mehanizam za horizontalno skaliranje, strategije replikacije i upravljanje konzistentnošću kroz Read i Write Concern opcije, backup strategije, kao i alate za praćenje i dijagnostiku sistema.

2 Osnove visoke dostupnosti u distribuiranim sistemima

2.1 Definicija i koncepti

Visoka dostupnost se definiše kao procenat vremena u kome je sistem dostupan za korišćenje u određenom vremenskom periodu. Standardna formulacija je "broj devetki- - sistem sa dostupnošću od 99,9% (tri devetke) tj. dozvoljivi prekid od oko 8,7 sati godišnje, dok sistem sa 99,999% (pet devetki) dopušta samo oko 5,25 minuta neplaniranog prekida godišnje [1].

U distribuiranim sistemima, visoka dostupnost se postiže eliminacijom pojedinačnih tačaka kvara (Single Points of Failure, SPOF) kroz redundantnost - uvođenjem višestrukih kopija podataka i višestrukih čvorova koji mogu preuzeti uloge u slučaju kvara. Ovaj pristup je osnova MongoDB arhitekture [2].

2.2 CAP teorema i njegova primena u MongoDB-u

CAP teorema, kojeg je formulisao Eric Brewer, tvrdi da distribuirani sistem ne može istovremeno da garantuje sva tri sledeća svojstva [1]:

- **Konzistentnost** (Consistency) - svi čvorovi vide iste podatke u isto vreme
- **Dostupnost** (Availability) - svaki zahtev dobija odgovor (bez garancije da sadrži najnovije podatke)
- **Tolerancija particije** (Partition Tolerance) - sistem nastavlja da radi i u slučaju gubitka mrežnih poruka između čvorova

MongoDB, kao i većina modernih distribuiranih baza podataka, bira toleranciju particije kao neophodan uslov (jer particije u distribuiranom sistemu nisu opcione - one se dešavaju). U normalnom radu MongoDB je konzistentan sistem (CP), ali nudi konfigurabilne mehanizme (Read/Write Concern) koji omogućavaju prilagođavanje između konzistentnosti i dostupnosti u zavisnosti od konkretnih zahteva aplikacije [3].

2.3 Replikacija kao osnova visoke dostupnosti

Replikacija je fundamentalni mehanizam visoke dostupnosti koji podrazumeva čuvanje više kopija podataka na različitim čvorovima. Replikacija ima dve ključne uloge [1]:

1. **Povećanje dostupnosti** - ukoliko jedan čvor nije dostupan, drugi čvorovi mogu preuzeti kontrolu i obradu zahteva
2. **Poboljšanje performansi čitanja** - zahtevi za čitanjem mogu biti distribuirani između više replika, čime se smanjuje opterećenje primarnog čvora

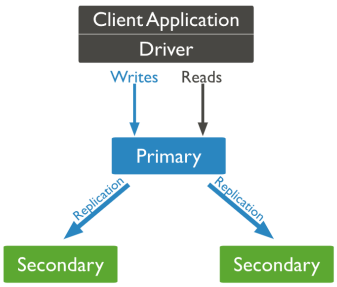
Osnovna dilema pri replikaciji je konzistentnost: kada se podatak menja na primarnom čvoru, koliko brzo i pouzdano ta promena treba da se propagira na sekundarne čvorove? MongoDB rešava ovaj problem kroz konfigurabilan Write Concern i asinhrone oplog (operations log) mehanizme.

3 Replica setovi

3.1 Arhitektura replica set-a

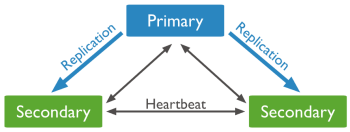
Replica set je osnovna jedinica visoke dostupnosti u MongoDB-u. Sastoji se od skupa mongod instanci koje čuvaju iste podatke. Tipičan replica set ima tri ili više članova [4]:

- **Primarni čvor** (Primary) - jedini član koji prihvata operacije pisanja. Sve write operacije se beleže u oplog (operations log) primarnog čvora.



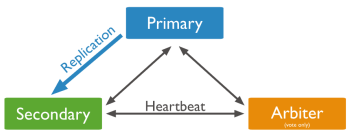
Slika 1: Prikaz rada primarnog cvora

- **Sekundarni čvorovi** (Secondaries) - replikuju oplog primarnog čvora i primenjuju operacije nad sopstvenim skupom podataka. Može da izvršava read operacije ukoliko je to konfigurisano.



Slika 2: Prikaz rada sekundarnog cvora

- **Arbitar** - specijalni član koji ne čuva kopiju podataka, već učestvuje samo u glasanju pri izboru novog primarnog čvora. Koristi se za postizanje neparnog broja glasova uz minimalne troškove infrastrukture.



Slika 3: Prikaz rada arbitra

Minimalni Replica setovi za produkcijsku upotrebu sastoji se od tri člana: dva punopravna člana (primarni i jedan sekundarni) i jednog arbitra, ili tri punopravna člana. Ovo obezbeđuje kvorum (većinu glasova) neophodan za izbor primarnog čvora [2].

3.2 Oplog (Operations Log)

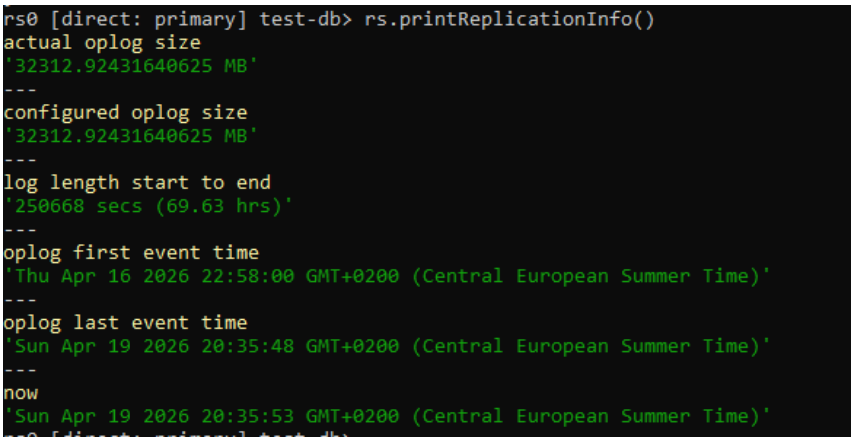
Oplog je specijalna obnovljiva kolekcija koja se čuva u lokalnoj bazi podataka svakog člana replica set-a. Primarni čvor beleži sve operacije koje menjaju podatke u oplogu. Sekundarni čvorovi asinhrono čitaju i primenjuju ove operacije na vlastite skupove podataka [4].

Ključne karakteristike oploga su:

- **Idempotentnost** - svaka operacija u oplogu može biti primenjena više puta bez promene rezultata, što omogućava pouzdano ponovno izvršavanje u slučaju prekida
- **Veličina** - veličina oploga direktno utiče na replication lag koji sistem može tolerisati; veći oplog omogućava sekundarnim čvorovima da zaostaju više bez potrebe za inicijalnom sinhronizacijom
- **Replikacioni prozor** - period za koji oplog čuva zapise; ukoliko sekundarni čvor zaostane više od tog perioda, neophodna je inicijalna sinhronizacija (initial sync)

```
// Pregled informacija o oplogu
use local
db.oplog.rs.find().sort({$natural: -1}).limit(5)

// Procena preostalih sati oploga
rs.printReplicationInfo()

// Pregled zaostajanja sekundarnih čvorova
rs.printSecondaryReplicationInfo()
```

Listing 1: Pregled stanja oploga



Slika 4: Prikaz procene preostalih sati oploga i zauzete memorije

3.3 Inicijalna sinhronizacija

Kada se novi čvor dodaje u replica set ili kada sekundarni čvor previše zaostane za primarnim, pokreće se inicijalna sinhronizacija (initial sync). Ovaj proces kopira sve podatke sa primarnog (ili drugog sinhronizovanog) čvora i primenjuje operacije iz oploga koje su se desile za vreme trajanja kopiranja [4].

```
// Pregled statusa replikacije
rs.status()

// Detaljan pregled - inicijalna sinhronizacija
db.adminCommand({ replSetGetStatus: 1, initialSync: 1 })
```

Listing 2: Praćenje napretka inicijalne sinhronizacije

3.4 Konfiguracija replica set-a

```
// Pokretanje mongod instanci sa --replSet parametrom:
mongod --replSet "rs0" --port 27017 --dbpath /data/db1 --bind_ip
    localhost
mongod --replSet "rs0" --port 27018 --dbpath /data/db2 --bind_ip
    localhost

// Dodavanje novog sekundarnog čvora
rs.add("localhost:27018")

// Dodavanje arbitra
rs.addArb("localhost:27021")

// Pregled trenutne konfiguracije
rs.conf()

// Pregled statusa svih članova
rs.status()
```

Listing 3: Inicijalizacija i konfiguracija replica set-a

Slika 5: Prikaz pokretanja sekundarnog cvora

```
  },
  {
    _id: 2,
    name: 'localhost:27018',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 112,
    optime: { ts: Timestamp({ t: 1776625620, i: 3 }), t: Long('3') },
    optimeDurable: { ts: Timestamp({ t: 1776625620, i: 3 }), t: Long('3') },
    optimeWritten: { ts: Timestamp({ t: 1776625620, i: 3 }), t: Long('3') },
    optimeDate: ISODate('2026-04-19T19:07:00.000Z'),
    optimeDurableDate: ISODate('2026-04-19T19:07:00.000Z'),
    optimeWrittenDate: ISODate('2026-04-19T19:07:00.000Z'),
    lastAppliedWallTime: ISODate('2026-04-19T19:07:00.205Z'),
    lastDurableWallTime: ISODate('2026-04-19T19:07:00.205Z'),
    lastWrittenWallTime: ISODate('2026-04-19T19:07:00.205Z'),
    lastHeartbeat: ISODate('2026-04-19T19:07:05.310Z'),
    lastHeartbeatRecv: ISODate('2026-04-19T19:07:05.960Z'),
    pingMs: Long('0'),
    lastHeartbeatMessage: '',
    syncSourceHost: '127.0.0.1:27017',
    syncSourceId: 0,
    infoMessage: '',
    configVersion: 4,
    configTerm: 3
  }
],
ok: 1,
```

Slika 6: Promena statusa sekundarnog cvora - prelazak u pokrenuto stanje

```
    },
    {
      _id: 3,
      name: 'localhost:27021',
      health: 1,
      state: 7,
      stateStr: 'ARBITER',
      uptime: 5,
      lastHeartbeat: ISODate('2026-04-19T19:11:42.887Z'),
      lastHeartbeatRecv: ISODate('2026-04-19T19:11:42.890Z'),
      pingMs: Long('0'),
      lastHeartbeatMessage: '',
      syncSourceHost: '',
      syncSourceId: -1,
      infoMessage: '',
      configVersion: 5,
      configTerm: 3
    }
  ],
```

Slika 7: Dodavanje arbitra u replica set

3.5 Parametri

Svaki član replica set-a ima konfigurabilne parametre koji utiču na ponašanje sistema [4]:

- **priority** - vrednost od 0 do 1000 koja utiče na verovatnoću da član postane primarni. Vrednost 0 označava da član nikada ne može postati primarni (skriveni čvor ili odloženi čvor).
- **votes** - broj glasova koje član ima u izbornom procesu (0 ili 1)
- **hidden** - skriveni čvor nije vidljiv aplikacijama i ne može postati primarni, ali učestvuje u glasanju; korisno za izolovanje analitičkih upita

- **secondaryDelaySecs** - odloženi čvor primenjuje operacije sa zakašnjenjem, što pruža zaštitu od slučajnog brisanja ili oštećenja podataka
- **tags** - oznake koje omogućavaju podešavanje Read Preference i Write Concern (npr. geografska lokacija čvora)

```
// Postavljanje prioriteta i odloženog čvora
cfg = rs.conf()
cfg.members[2].priority = 0
cfg.members[2].hidden = true
// Odloženi čvor - primenjuje operacije sa zakašnjenjem od 1 sata
cfg.members[2].secondaryDelaySecs = 3600
rs.reconfig(cfg)

// Postavljanje tagova za deployment koji uzima u obzir geolokaciju
cfg = rs.conf()
cfg.members[0].tags = { "dc": "us-east", "rack": "rack1" }
cfg.members[1].tags = { "dc": "us-west", "rack": "rack1" }
cfg.members[2].tags = { "dc": "eu-central", "rack": "rack1" }
rs.reconfig(cfg)
```

Listing 4: Konfiguracija prioriteta i skrivenih čvorova

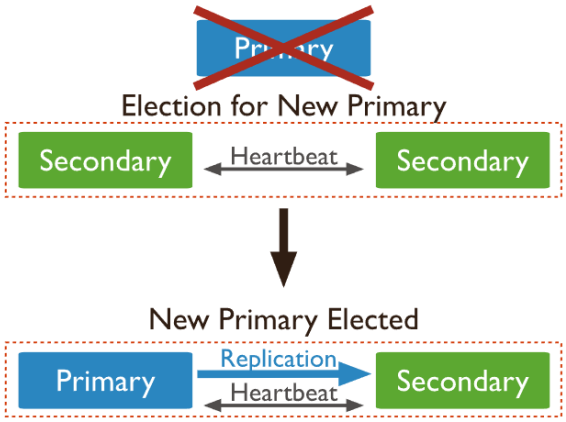
```
rs0 [direct: primary] test> cfg = rs.conf()
... cfg.members.forEach(m => print(m._id, m.host, JSON.stringify(m.tags), "priority:", m.priority, "delay:", m.secondaryD
laySecs))
0 127.0.0.1:27017 {"dc":"us-east","rack":"rack1"} priority: 1 delay: Long('0')
1 localhost:27020 {"dc":"us-west","rack":"rack1"} priority: 1 delay: Long('0')
2 localhost:27018 {"dc":"eu-central","rack":"rack1"} priority: 0 delay: Long('120')
3 localhost:27021 {} priority: 0 delay: Long('0')
```

Slika 8: Dodavanje tagova i konfiguracija odloženog čvora

4 Automatski failover i izbor primarnog čvora

4.1 Mehanizam detekcije kvara

Članovi replica set-a međusobno komuniciraju slanjem periodičnih heartbeat poruka svakih dve sekunde. Ukoliko primarni čvor ne odgovori na heartbeat u periodu od 10 sekundi (electionTimeoutMillis), sekundarni čvorovi zaključuju da je primarni nedostupan i pokreću proces izbora [5].



Slika 9: Prikaz procesa izbora primarnog čvora

4.2 Protokol izbora (Election Protocol)

MongoDB koristi modifikovanu varijantu Raft koncenzus algoritma za izbor primarnog čvora [6]. Proces izbora odvija se na sledeći način:

1. **Detekcija** - sekundarni čvor koji detektuje gubitak primarnog čvora (šaljući heartbeat poruke primarnom čvoru); ukoliko čvor ne dobije odgovor u roku od 10 sekundi, primarni čvor se smatra nedostupnim
2. **Nominacija** - kandidat šalje zahtev za glasanjem svim ostalim vidljivim članovima skupa
3. **Glasanje** - svaki član glasa za kandidata koji ima najažurniji oplog i koji nije glasao za drugog kandidata u trenutnom izboru
4. **Izbor** - kandidat koji dobije glasove od apsolutne većine vidljivih glasačkih članova postaje novi primarni
5. **Potvrda** - novi primarni šalje heartbeat svim članovima obaveštavajući ih o svom novom statusu

Čvorovi sa višim prioritetom imaju prednost u procesu glasanja - ukoliko je čvor sa višim prioritetom ažurniji od tekućeg kandidata, on može inicirati novi krug glasanja [5].

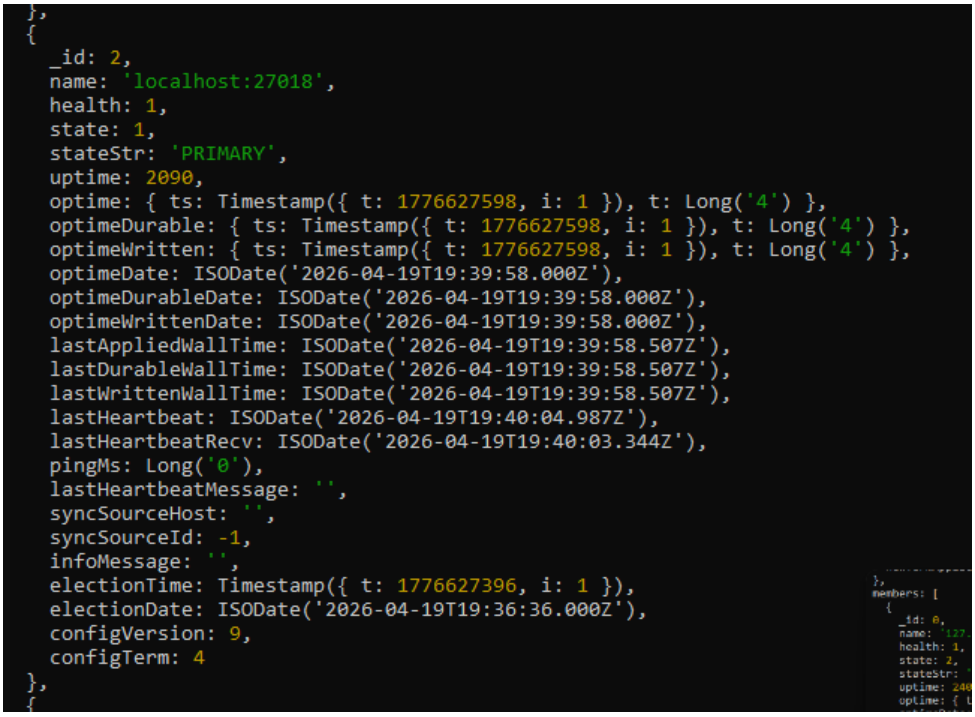
4.3 Vreme failover-a

Tipično vreme potrebno za automatski failover u MongoDB-u je između 10 i 30 sekundi, u zavisnosti od konfiguracije i mrežnih uslova. Ovo vreme uključuje [2]:

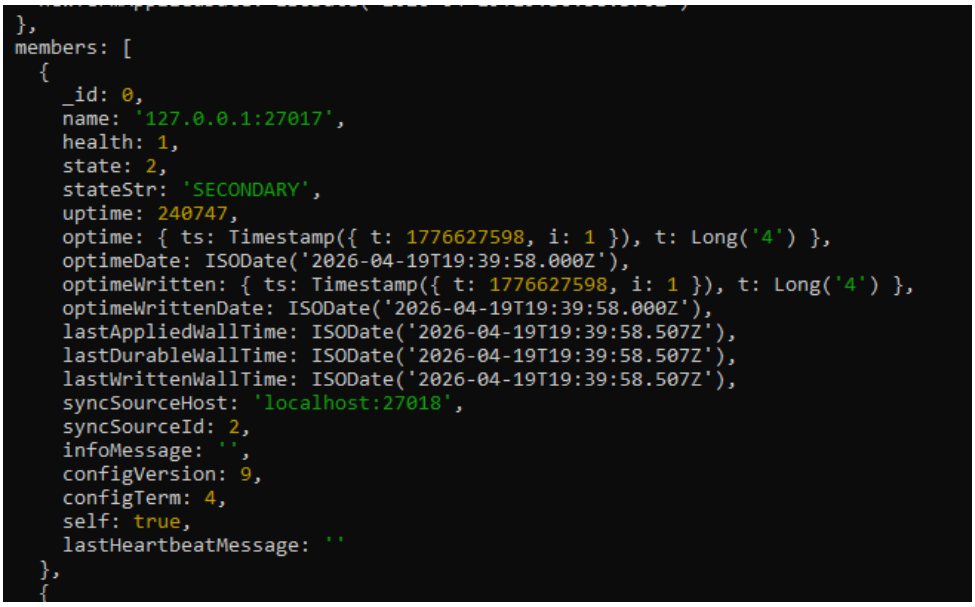
- Detekcija kvara primarnog čvora: do `electionTimeoutMillis`
- Proces glasanja i izbora: obično 1–5 sekundi
- Vreme koje nova primarna instanca treba da počne prihvata pisanja: nekoliko sekundi

```
// Podešavanje timeout-a za glasanje
cfg = rs.conf()
cfg.settings = {
  heartbeatTimeoutSecs: 10,
  electionTimeoutMillis: 10000,
  catchUpTimeoutMillis: -1  // neograničeno hvatanje
}
rs.reconfig(cfg)

// Simulacija failover-a (step down primarnog čvora)
// Na primarnom čvoru:
rs.stepDown()  // preda ulogu primarnog za 60 sekundi

// Provera ko je trenutno primarni čvor
db.hello()
```

Listing 5: Konfiguracija parametara glasanja i failover-a



Slika 10: Prethodno sekundarni cvor, nakon failover-a postaje primarni



```
},
members: [
  {
    _id: 0,
    name: '127.0.0.1:27017',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 240747,
    optime: { ts: Timestamp({ t: 1776627598, i: 1 }), t: Long('4') },
    optimeDate: ISODate('2026-04-19T19:39:58.000Z'),
    optimeWritten: { ts: Timestamp({ t: 1776627598, i: 1 }), t: Long('4') },
    optimeWrittenDate: ISODate('2026-04-19T19:39:58.000Z'),
    lastAppliedWallTime: ISODate('2026-04-19T19:39:58.507Z'),
    lastDurableWallTime: ISODate('2026-04-19T19:39:58.507Z'),
    lastWrittenWallTime: ISODate('2026-04-19T19:39:58.507Z'),
    syncSourceHost: 'localhost:27018',
    syncSourceId: 2,
    infoMessage: '',
    configVersion: 9,
    configTerm: 4,
    self: true,
    lastHeartbeatMessage: ''
  },
  {
```

Slika 11: Nakon failover-a, i ponovnog pokretanja primarnog cvora, on postaje sekundarni

4.4 Read i Write Concern u kontekstu visoke dostupnosti

Read i Write Concern su mehanizmi koji omogućavaju aplikacijama da garantuju konzistentnost i trajnost podataka, i direktno utiču na ponašanje sistema u slučaju failover-a [7].

4.4.1 Write Concern

Write Concern definiše na koliko članova replica set-a operacija pisanja mora biti potvrđena pre nego što se klijentu vrati odgovor o uspešnosti:

Tabela 1: Write Concern opcije i njihov uticaj na dostupnost

Write Concern	Potvrda	Uticaj na dostupnost
`w: 0`	Bez potvrde	Najbrže, mogući gubitak podataka
`w: 1`	Potvrda samo od primarnog	Podrazumevano; gubitak podataka u slučaju kvara pre replikacije
`w: "majority"`	Potvrda od većine glasačkih članova	Bez gubitka potvrđenih podataka- sporije
`w: N`	Potvrda od N članova	Prilagodljivo
`j: true`	Zapisivanje u journal pre potvrde	Zaštita od pada procesa

```
// Pisanje sa potvrdom od većine članova i journal opcijom
db.narudzbine.insertOne(
  { kupacId: "K001", iznos: 15000, datum: new Date() },
  { writeConcern: { w: "majority", j: true, wtimeout: 5000 } }
)

```

```
// Pisanje sa potvrdom od 2 člana
db.proizvodi.updateOne(
  { _id: "P001" },
  { $set: { cena: 2999 } },
  { writeConcern: { w: 2 } }
)
```

Listing 6: Primena Write Concern opcija

4.4.2 Read Concern

Read Concern kontroliše konzistentnost podataka koji se vraćaju pri operacijama čitanja [3]:

Tabela 2: Read Concern opcije

Read Concern	Opis
`local`	Podrazumevano; vraća najnovije podatke dostupne čvoru, bez garancije da su potvrđeni od strane većine
`available`	Slično kao local, ali bez garancija u slučaju particije
`majority`	Vraća samo podatke potvrđene od strane većine; zaštita od vraćanja podataka koji će biti povučeni u slučaju failover-a
`linearizable`	Najjača garancija; čitanje vraća podatke koji odražavaju sve prethodne uspešne write operacije; može biti sporo
`snapshot`	Čitanje konzistentne snimke baze u okviru transakcije

4.4.3 Read Preference

Read Preference definiše sa kojih članova replica set-a aplikacija čita podatke [8]:

- **primary** (podrazumevano) - čitanje samo sa primarnog; garantuje najsvežije podatke
- **primaryPreferred** - čitanje sa primarnog ako je dostupan, inače sa sekundarnog
- **secondary** - čitanje samo sa sekundarnih čvorova; smanjuje opterećenje primarnog, ali može vraćati zastarele podatke
- **secondaryPreferred** - preferira sekundarne čvorove; čita sa primarnog samo ako nema dostupnih sekundarnih
- **nearest** - čitanje sa čvora sa najnižom latencijom, bez obzira na tip

```
// Čitanje sa sekundarnih čvorova za analitičke upite
db.getMongo().setReadPref("secondary")

// Čitanje sa najbližeg čvora sa tagom za data center
db.getMongo().setReadPref("nearest", [{ "dc": "us-east" }])

// U connection stringu:
// mongodb://mongo1,mongo2,mongo3/?replicaSet=rs0&readPreference=
   secondaryPreferred
```

Listing 7: Konfiguracija Read Preference

5 Shardovani klasteri

5.1 Arhitektura shardovanih klastera

Dok replica setovi obezbeđuju visoku dostupnost kroz redundantnost podataka, shardovani klasteri pružaju horizontalnu skalabilnost kroz distribuciju podataka na više Replica set-ova (shardova). [9].

Shardovani klaster se sastoji od tri komponente:

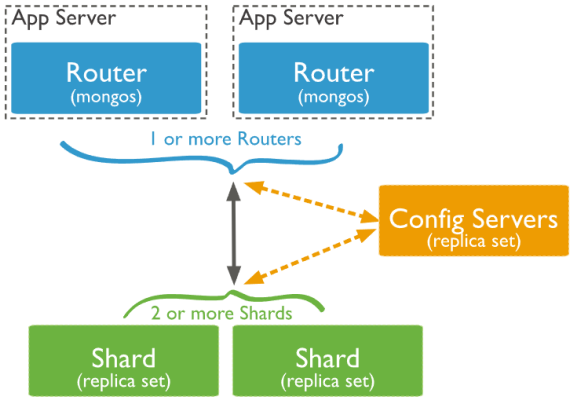
- **Shardovi** - svaki shard je replica set koji čuva podskup podataka kolekcije
- **Config Server** - čuvaju metapodatke o klasteru: mapiranje dela opsega shard ključa (chunk-ova) na konkretne shardove, listu shardova i konfiguraciju balansera. U produkciji, Config Servers su uvek replica set (CSRS - Config Server replica set) sa tri člana
- **mongos** - query router koji prima zahteve od aplikacija, konsultuje Config Servers i prosleđuje upite odgovarajućim shardovima; ne čuva podatke i može ih biti više za visoku dostupnost

5.2 Shard ključ i distribucija podataka

Shard ključ je polje (ili kombinacija polja) u dokumentu prema kome se podaci distribuiraju između shardova. Izbor shard ključa je jedna od najvažnijih odluka pri projektovanju shardovanih klastera, jer direktno utiče na performanse, distribuciju podataka i mogućnost skaliranja [2].

Dobar shard ključ treba da ima sledeće karakteristike [9]:

- **Visoka kardinalnost** - dovoljan broj različitih vrednosti da se podaci mogu ravnomerno rasporediti; ključ sa malo različitih vrednosti ograničava maksimalni broj chunk-ova
- **Niska učestalost** - vrednosti shard ključa ne treba da se javljaju previše često; ako veliki broj dokumenata ima istu vrednost shard ključa, ti dokumenti moraju biti na istom shardu (jumbo chunk)
- **Monotona promenljivost** - izbegavati monotono rastuće vrednosti (npr. ObjectId ili timestamp) kao jedini shard ključ, jer sve operacije pisanja idu na isti shard dok se chunk ne podeli; koristiti hešovani shard ključ za ravnomerniju distribuciju pisanja



Slika 12: Prikaz rada shard-a

```
// Omogućavanje sharding-a za bazu podataka
sh.enableSharding("test-db")

// Šardovanje kolekcije po hešovanom ključu
// (za ravnomernu distribuciju insert operacija)
sh.shardCollection("testdb.users", { _id: "hashed" })

// Dodavanje podataka u kolekciji users
db.users.insertMany([
    { name: "Alice", city: "NYC" },
    { name: "Bob", city: "LA" },
    { name: "Charlie", city: "London" },
    { name: "Diana", city: "Berlin" },
    { name: "Eve", city: "Paris" }
])
// Pregled stanja klastera
sh.status()

// Pregled distribucije chunk-ova po shardovima
db.adminCommand({ listShards: 1 })

// Dodavanje drugog shard-a
sh.addShard("rs1/localhost:27040")
```

Listing 8: Konfiguracija Sharding-a

```
switched to db testdb
[direct: mongos] testdb> db.users.getShardDistribution()
Shard rs1 at rs1/localhost:27040
{
  data: '0B',
  docs: 0,
  chunks: 1,
  'estimated data per chunk': '0B',
  'estimated docs per chunk': 0
}
---
Totals
{
  data: '0B',
  docs: 0,
  chunks: 1,
  'Shard rs1': [ '0 % data', '0 % docs in cluster', '0B avg obj size on shard' ]
}
```

Slika 13: Inicijalizacija shardovanog klastera - pre dodavanja podataka

```
}
[direct: mongos] testdb> db.users.getShardDistribution()
Shard rs1 at rs1/localhost:27040
{
  data: '265B',
  docs: 5,
  chunks: 1,
  'estimated data per chunk': '265B',
  'estimated docs per chunk': 5
}
---
Totals
{
  data: '265B',
  docs: 5,
  chunks: 1,
  'Shard rs1': [
    '100 % data',
    '100 % docs in cluster',
    '53B avg obj size on shard'
  ]
}
```

Slika 14: Inicijalizacija shardovanog klastera - nakon dodavanja podataka

```
]
[direct: mongos] testdb> db.adminCommand({ listShards: 1 })
{
  shards: [
    {
      _id: 'rs1',
      host: 'rs1/localhost:27040',
      state: 1,
      topologyTime: Timestamp({ t: 1776631116, i: 32 }),
      replSetConfigVersion: Long('-1')
    },
    {
      _id: 'rs2',
      host: 'rs2/localhost:27041',
      state: 1,
      topologyTime: Timestamp({ t: 1776631461, i: 32 }),
      replSetConfigVersion: Long('-1')
    }
  ],
```

Slika 15: Dodavanje drugog shard-a

```
[direct: mongos] testdb> db.users.getShardDistribution()
Shard rs1 at rs1/localhost:27040
{
  data: '45KiB',
  docs: 1035,
  chunks: 1,
  'estimated data per chunk': '45KiB',
  'estimated docs per chunk': 1035
}
---
Shard rs2 at rs2/localhost:27041
{
  data: '45KiB',
  docs: 519,
  chunks: 1,
  'estimated data per chunk': '45KiB',
  'estimated docs per chunk': 519
}
---
Totals
{
  data: '91KiB',
  docs: 1554,
  chunks: 2,
  'Shard rs1': [
    '49.91 % data',
    '66.6 % docs in cluster',
    '90B avg obj size on shard'
  ],
  'Shard rs2': [
    '50.08 % data',
    '33.39 % docs in cluster',
    '90B avg obj size on shard'
  ]
}
```

Slika 16: Distribucija podataka između shardova nakon naknadnog dodavanja podataka

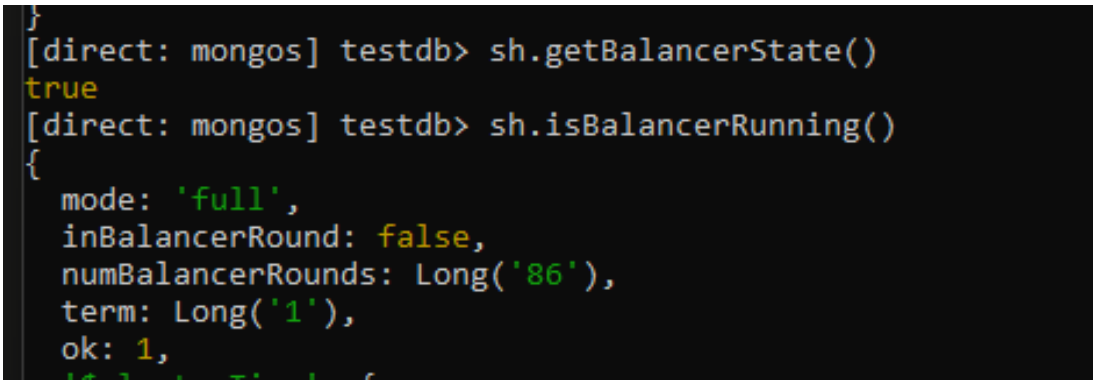
5.3 Balansiranje podataka

MongoDB automatski balansira podatke između shardova kroz proces migracije chunk-ova. Balancer je pozadinski proces koji radi na Config Servers i premešta chunk-ove sa preopterećenih shardova na manje opterećene [9].

```
// Provjera statusa balancera
sh.getBalancerState()
sh.isBalancerRunning()

// Privremeno zaustavljanje balancera (npr. tokom backup-a)
sh.stopBalancer()
sh.startBalancer()

// Definisanje vremenskog prozora za balansiranje
// (samo noću kako bi se smanjio uticaj na produkciju)
db.settings.updateOne(
  { _id: "balancer" },
  { $set: {
      activeWindow: { start: "23:00", stop: "06:00" }
```

```
  }},
  { upsert: true }
)

// Ručno pokretanje migracije chunk-a
db.adminCommand({
    moveChunk: "testdb.users",
    find: { _id: 1 },
    to: "rs2"
})
```

Listing 9: Upravljanje balancerom



Slika 17: Provera statusa balancera i njegovog pokretanja

5.4 Targeted vs. Broadcast operacije

Efikasnost upita u shardovanom klasteru zavisi od toga da li mongos može da odredi na kojem shard-u da prosledi upit:

- **Targeted (usmerene) operacije** - upit sadrži shard ključ u filter uslovu, mongos prosleđuje upit samo odgovarajućim shardovima; visoka efikasnost
- **Broadcast operacije** - upit ne sadrži shard ključ, mongos mora da prosledi upit svim shardovima i spoji rezultate (scatter-gather); skupe operacije koje treba minimizovati

```
// Targeted upit - sadrži shard ključ (kupacId)
db.narudzbine.find({ kupacId: "K001", datum: { $gte: new Date("
    2026-01-01") } })
  .explain("executionStats")

// Broadcast upit - ne sadrži shard ključ
// Svaki shard mora biti upitan
db.narudzbine.find({ iznos: { $gt: 10000 } })
  .explain("executionStats")
// U explain izlazu: "SHARD_MERGE" ukazuje na broadcast operaciju
```

Listing 10: Analiza targeted vs. broadcast upita

6 Backup i strategije oporavka

6.1 Značaj backup-a u HA okruženju

Replica setovi pružaju zaštitu od kvarova hardvera i mrežnih problema, ali ne zamenjuju backup. Backup je neophodan za zaštitu od:

- Slučajnog brisanja podataka ili kolekcija (operacija se replicira na sve sekundarne čvorove)
- Oštećenja podataka uzrokovanog greškama u aplikaciji
- Ransomware napada

Prava visoka dostupnost kombinuje replikaciju za brzi oporavak od hardverskih kvarova sa redovnim backup-ovima za oporavak od logičkih grešaka [1].

6.2 mongodump i mongorestore

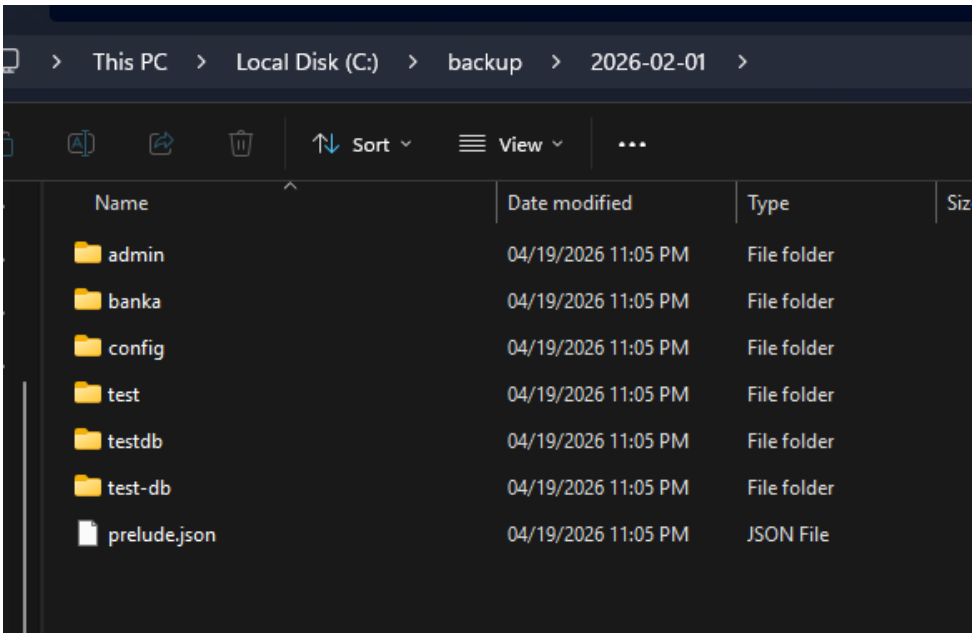
`mongodump` je alat koji kreira logički backup (BSON dump) baze podataka. Pogodan je za manje baze podataka i testna okruženja, ali za velike produkcijske baze preporučuju se brže metode [10].

```
// Backup cele instance
mongodump --host mongo1:27017 --out /backup/2026-02-01

// Backup specifične baze uz kompresiju
mongodump --host rs0/mongo1:27017,mongo2:27018 \
  --db prodajnaBaza \
  --gzip \
  --out /backup/prodajnaBaza-2026-02-01

// Backup specifične kolekcije
mongodump --host rs0/mongo1:27017 \
  --db prodajnaBaza \
  --collection narudzbine \
  --query '{"datum": {"$gte": {"$date": "2026-01-01T00:00:00Z"}}}' \
  --out /backup/

// Restore cele baze
mongorestore --host mongo1:27017 \
  --drop \
  /backup/2026-02-01

// Restore specifične kolekcije uz gzip
mongorestore --host rs0/mongo1:27017 \
  --gzip \
  --db prodajnaBaza \
  --collection narudzbine \
  /backup/prodajnaBaza-2026-02-01/prodajnaBaza/narudzbine.bson.gz
```

Listing 11: mongodump i mongorestore

```
2026-04-19T23:05:58.784+0200    done dumping `banka.transakcije` (3 documents)
2026-04-19T23:05:58.785+0200    writing `test.narudzbine` to `C:\backup\2026-02-01\test\narudzbine.bson`
2026-04-19T23:05:58.785+0200    writing `banka.kriticni_podaci` to `C:\backup\2026-02-01\banka\kriticni_podaci.bson`
2026-04-19T23:05:58.786+0200    done dumping `config.settings` (1 document)
2026-04-19T23:05:58.786+0200    done dumping `test-db.racuni` (2 documents)
2026-04-19T23:05:58.786+0200    writing `test-db.korisnici` to `C:\backup\2026-02-01\test-db\korisnici.bson`
2026-04-19T23:05:58.786+0200    writing `test-db.istorija` to `C:\backup\2026-02-01\test-db\istorija.bson`
2026-04-19T23:05:58.786+0200    done dumping `banka.kriticni_podaci` (0 documents)
2026-04-19T23:05:58.787+0200    writing `test-db.transakcije` to `C:\backup\2026-02-01\test-db\transakcije.bson`
2026-04-19T23:05:58.787+0200    done dumping `test.narudzbine` (1 document)
2026-04-19T23:05:58.787+0200    writing `test-db.sesije` to `C:\backup\2026-02-01\test-db\sesije.bson`
2026-04-19T23:05:58.788+0200    done dumping `test-db.korisnici` (0 documents)
2026-04-19T23:05:58.789+0200    writing `banka.sesije` to `C:\backup\2026-02-01\banka\sesije.bson`
2026-04-19T23:05:58.790+0200    done dumping `test-db.istorija` (0 documents)
2026-04-19T23:05:58.790+0200    writing `test-db.logovi` to `C:\backup\2026-02-01\test-db\logovi.bson`
2026-04-19T23:05:58.790+0200    done dumping `test-db.transakcije` (0 documents)
2026-04-19T23:05:58.790+0200    writing `banka.narudzbine` to `C:\backup\2026-02-01\banka\narudzbine.bson`
2026-04-19T23:05:58.790+0200    done dumping `test-db.sesije` (0 documents)
2026-04-19T23:05:58.791+0200    done dumping `banka.sesije` (0 documents)
2026-04-19T23:05:58.791+0200    done dumping `banka.narudzbine` (0 documents)
2026-04-19T23:05:58.791+0200    done dumping `test-db.logovi` (0 documents)
```

Slika 18: Backup kreiran mongodump-om



Slika 19: Backup fajlovi kreirani mongodump-om

6.3 Point-in-Time Recovery

Point-in-Time Recovery (PITR) omogućava vraćanje baze podataka u stanje u koje je bila u bilo kom trenutku unutar prozora za oporavak. MongoDB Atlas pruža PITR ugrađenu kao uslugu, dok se u samoupravljivim okruženjima postiže kombinacijom periodičnih snapshot backup-ova i čuvanja oploga [10].

Strategija za PITR u samoupravljivim okruženjima:

1. Periodično (npr. dnevno) kreirati filesystem snapshot sa pauziranim I/O operacijama
2. Čuvati oplog koji pokriva period između snapshot-ova
3. Za oporavak: povratiti poslednji snapshot, zatim primeniti operacije iz oploga do željenog trenutka

```
// Pauziranje write operacija za konzistentni snapshot diska
// Na sekundarnom čvoru (da ne utiče na produkciju):
db.fsyncLock()

// Odbravljivanje nakon snapshot-a
db.fsyncUnlock()
```

Listing 12: Pauziranje I/O operacija za konzistentni snapshot

6.4 Odloženi sekundarni čvorovi kao zaštita od grešaka

Odloženi (delayed) sekundarni čvorovi primenjuju operacije iz oploga sa konfigurisanim zakašnjenjem. Na primer, čvor sa zakašnjenjem od jednog sata reflektovaće stanje baze kakvo je bilo sat vremena ranije. Ovo pruža zaštitu od slučajnog brisanja ili oštećenja - administrator ima vremenski prozor da detektuje problem i inicira oporavak sa odloženog čvora pre nego što i on primeni štetnu operaciju [2].

```
cfg = rs.conf()
// Treći član (index 2) postavljamo kao odloženi čvor
cfg.members[2].priority = 0        // ne može postati primarni
cfg.members[2].hidden = true       // nevidljiv aplikacijama
cfg.members[2].secondaryDelaySecs = 3600  // 1 sat zakašnjenja
rs.reconfig(cfg)
```

Listing 13: Konfiguracija odloženog sekundarnog čvora

7 Monitoring i dijagnostika

7.1 Značaj monitoringa za visoku dostupnost

Reaktivan pristup visoke dostupnosti (čekanje na kvar pa oporavak) nije dovoljan za kritične sisteme. Proaktivno praćenje metrika sistema omogućava detekciju potencijalnih problema pre nego što dovedu do pada sistema [11]. MongoDB pruža više nivoa alata za praćenje stanja sistema [12].

7.2 serverStatus i mongostat

```
// Opšte stanje servera
db.serverStatus()

// Metrike replikacije
db.serverStatus().repl
db.serverStatus().opcounters

// Replication lag sekundarnih čvorova
rs.status().members.forEach(function(m) {
  if (m.state === 2) {  // 2 = SECONDARY
    print(m.name + " lag: " +
      (rs.status().date - m.optimeDate) / 1000 + "s")
  }
})
```

Listing 14: Ključne metrike za monitoring HA

7.3 Praćenje replica set stanja

```
// Detaljni status svih članova
rs.status()

// Pregled ko je trenutno primarni
db.hello()

// Provera replikacionog zaostajanja
// (u sekundama za svaki sekundarni čvor)
var status = rs.status()
status.members.forEach(function(member) {
  if (member.stateStr !== "PRIMARY") {
    var lag = (status.members[0].optimeDate - member.optimeDate) / 1000
    print(member.name + ": " + lag + " sekundi zaostajanja")
  }
})

// Pregled operacija koje se izvršavaju u realnom vremenu
db.currentOp({ active: true, secs_running: { $gt: 5 } })
```

Listing 15: Monitoring replica set stanja

7.4 Preporučene metrike za alerting

Za efikasan monitoring HA sistema, važno je definisati pragove za upozorenja pre nego što nastanu kvarovi [1]. U MongoDB kontekstu, ključne metrike za alerting su [11]:

Tabela 3: Ključne metrike za monitoring visoke dostupnosti

Metrika	**Opis**	**Prag upozorenja**
Replication Lag	Zaostajanje sekundarnih čvorova za primarnim	> 10 sekundi
Replica Set State	Status svakog člana (PRIMARY/SECONDARY/etc.)	Bilo koji čvor nije u očekivanom stanju
Oplog Window	Preostalo vreme u oplogu	< 24 sata
Connections	Broj aktivnih konekcija	> 80% maksimuma
Page Faults	Broj straničnih grešaka	Naglo povećanje
Queue Length	Dužina reda čekanja za čitanje/pisanje	> 0 tokom dužeg perioda
Disk IOPS	Iskorišćenost diskova	> 80% kapaciteta

8 Geografski distribuirani deployment

8.1 Multi-data-center replica set

Za zaštitu od prirodnih katastrofa koje zahvataju čitav data centar (požar, poplava, nestanak struje), MongoDB replica set može biti rasprostranjen na više geografskih lokacija. Ključni izazov je konfigurisanje sistema tako da failover bude automatski i brz, a istovremeno da se izbegne tzv. split-brain situacija [2].

Postoje dve uobičajene konfiguracije za deployment u dva data centra:

Konfiguracija 1 — asimetrična zaštita (3 glasača):

- **Data centar A** (primarni): 2 punopravna člana (jedan primarni, jedan sekundarni)
- **Data centar B** (sekundarni): 1 punopravni član

Ukupno 3 glasača, kvorum = 2. U slučaju gubitka data centra B, data centar A ima 2 od 3 glasa i može da izabere novog primarnog. Međutim, u slučaju gubitka data centra A, data centar B ima samo 1 od 3 glasa i **ne može** samostalno da formira kvorum. Ova konfiguracija štiti od otkaza sekundarnog, ali ne i primarnog data centra.

Konfiguracija 2 — simetrična zaštita uz treću lokaciju (3 glasača):

- **Data centar A**: 1 punopravni član
- **Data centar B**: 1 punopravni član
- **Treća lokacija** (npr. oblak): 1 arbitar

Ukupno 3 glasača, kvorum = 2. U slučaju gubitka bilo kog data centra, preostali čvor zajedno sa arbitrom dostiže kvorum (2 od 3 glasa) i može izabrati novog primarnog. Ovo je preporučeni pristup kada je potrebna simetrična zaštita od otkaza jednog data centra.

8.2 Zonal tagging i Read Preference

Tag setovi (zone tagovi) omogućavaju Geographic-aware routing upita - aplikacije u određenoj geografskoj regiji mogu čitati sa lokalnih sekundarnih čvorova, čime se smanjuje latencija, dok se pisanje uvek odvija na primarnom čvoru bez obzira na lokaciju [8].

```
// Postavljanje tagova po data centrima
cfg = rs.conf()
cfg.members[0].tags = { "region": "us-east" }
cfg.members[1].tags = { "region": "us-east" }
cfg.members[2].tags = { "region": "eu-west" }
cfg.members[3].tags = { "region": "eu-west" }
rs.reconfig(cfg)

// Aplikacija u EU čita sa lokalnog sekundarnog čvora
// Primer u connection stringu:
// readPreference=secondary&readPreferenceTags=region:eu-west

// U kodu - čitanje sa regionalnog čvora
db.getMongo().setReadPref("secondary", [{ "region": "eu-west" }])
db.narudzbine.find({ status: "aktivna" })
```

Listing 16: Konfiguracija geografskog routinga

9 WiredTiger storage engine i journaling

9.1 Uloga storage engine-a u HA

MongoDB od verzije 3.2 koristi WiredTiger kao podrazumevani storage engine. WiredTiger direktno utiče na visoku dostupnost kroz mehanizme journalinga i kontrolnih tačaka (checkpoints) koji obezbeđuju trajnost podataka i omogućavaju brz oporavak u slučaju neočekivanog pada procesa [13].

9.2 Journaling

Journal je write-ahead log (WAL) koji WiredTiger koristi za osiguranje trajnosti operacija. Pre nego što WiredTiger primeni promenu na podatke u memoriji, promena se beleži u journal na disku. U slučaju neočekivanog pada, WiredTiger pri sledećem pokretanju čita journal i primenjuje sve zabeležene operacije koje nisu bile upisane u podatkovne fajlove, čime se obezbeđuje oporavak u konzistentnom stanju [13].

Ključne karakteristike journalinga u WiredTiger-u:

- **Kontrolne tačke** - WiredTiger kreira checkpoint (konzistentni snapshot podatkovnih fajlova) na svakih 60 sekundi ili kada journal dostigne 2 GB. Između checkpointa, journal čuva sve promene
- **Kompresija journala** - journal zapisi se kompresuju za smanjenje I/O operacija na disku
- **Sinhronizacija** - WiredTiger sinhronizuje journal na disk svakih 100 ms (podrazumevano)

```
// Pregled WiredTiger statistika
db.serverStatus().wiredTiger

// Specifično - metrike keša
db.serverStatus().wiredTiger.cache

// Konfiguracija veličine keša (u mongod.conf):
// storage:
//   wiredTiger:
//     engineConfig:
//       cacheSizeGB: 4

// Provera je li journaling aktivan
db.adminCommand({ getCmdLineOpts: 1 }).parsed.storage
```

Listing 17: Pregled WiredTiger metrika i podešavanja

10 Zaključak

Replica set je osnovna jedinica visoke dostupnosti u MongoDB-u. Automatski failover zasnovan na modifikovanom Raft konsenzus algoritmu obezbeđuje brzo preuzimanje uloge primarnog čvora. Oplog i mehanizmi replikacije garantuju da sekundarni čvorovi uvek imaju ažurnu kopiju podataka, dok odloženi sekundarni čvorovi pružaju zaštitu od logičkih grešaka.

MongoDB pruža sveobuhvatan skup mehanizama za postizanje visoke dostupnosti koji se mogu prilagoditi različitim zahtevima i budžetima - od jednostavnih tročlanih replica set-ova za manje aplikacije, do kompleksnih shardovanih klastera rasprostrtih u više geografskih regija za globalne produkcijske sisteme.

Konfigurabilni **Write Concern** i **Read Concern** mehanizmi daju aplikacijama preciznu kontrolu nad kompromisom između konzistentnosti, dostupnosti i performansi.Garantuje se da potvrđeni podaci neće biti izgubljeni čak ni u slučaju failover-a, dok Read Preference opcije omogućavaju distribuciju opterećenja čitanja na sekundarne čvorove.

Shardovani klasteri proširuju visoku dostupnost na horizontalno skalabilne sisteme, pri čemu svaki shard ostaje replica set koji sam po sebi pruža redundantnost podataka. Izbor shard ključa je kritična odluka koja direktno utiče na distribuciju podataka i efikasnost upita.

Back-up strategije dopunjuju replikaciju i pružaju zaštitu od logičkih grešaka i katastrofa koje zahvataju sve čvorove klastera.

WiredTiger storage engine sa mehanizmima journalinga i kontrolnih tačaka obezbeđuje trajnost podataka i brz oporavak od pada procesa. Pravilna konfiguracija veličine keša i kompresije optimizuje iskorišćenost resursa.

Konačno, posmatranje i monitoring ključnih metrika - replikacionog kašnjenja, statusa čvorova, veličine oploga i iskorišćenosti resursa - omogućava detekciju i rešavanje potencijalnih problema pre nego što dovedu do pada sistema, što je osnova svake istinski visoko dostupne arhitekture.

11 Literatura

[1] Raghu Ramakrishnan i Johannes Gehrke. *Database Management Systems*. 3rd. New York, NY: McGraw-Hill, 2003. ISBN: 978-0072465631.

[2] Shannon Bradshaw, Eoin Brazil i Kristina Chodorow. *MongoDB: The Definitive Guide*. 3rd. Sebastopol, CA: O'Reilly Media, 2019. ISBN: 978-1491954461.

[3] MongoDB, Inc. *Read Concern — MongoDB Manual*. MongoDB. 2025. URL: https://www.mongodb.com/docs/manual/reference/read-concern/ (posećeno 1. 2. 2026.).

[4] MongoDB, Inc. *Replication — MongoDB Manual*. MongoDB. 2025. URL: https://www.mongodb.com/docs/manual/replication/ (posećeno 1. 2. 2026.).

[5] MongoDB, Inc. *Replica Set Elections — MongoDB Manual*. MongoDB. 2025. URL: https://www.mongodb.com/docs/manual/core/replica-set-elections/ (posećeno 1. 2. 2026.).

[6] Diego Ongaro i John Ousterhout. „In Search of an Understandable Consensus Algorithm". U: *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC'14)* (2014.), str. 305–319.

[7] MongoDB, Inc. *Write Concern — MongoDB Manual*. MongoDB. 2025. URL: https://www.mongodb.com/docs/manual/reference/write-concern/ (posećeno 1. 2. 2026.).

[8] MongoDB, Inc. *Read Preference — MongoDB Manual*. MongoDB. 2025. URL: https://www.mongodb.com/docs/manual/core/read-preference/ (posećeno 1. 2. 2026.).

[9] MongoDB, Inc. *Sharding — MongoDB Manual*. MongoDB. 2025. URL: https://www.mongodb.com/docs/manual/sharding/ (posećeno 1. 2. 2026.).

[10] MongoDB, Inc. *Backup and Restore — MongoDB Manual*. MongoDB. 2025. URL: https://www.mongodb.com/docs/manual/core/backups/ (posećeno 1. 2. 2026.).

[11] Guy Harrison i Michael Harrison. *MongoDB Performance Tuning: Optimizing MongoDB Databases and their Applications*. 2nd. New York, NY: Apress, 2024. ISBN: 978-1484288528.

[12] MongoDB, Inc. *Monitoring for MongoDB — MongoDB Manual*. MongoDB. 2025. URL: https://www.mongodb.com/docs/manual/administration/monitoring/ (posećeno 1. 2. 2026.).

[13] MongoDB, Inc. *WiredTiger Storage Engine — MongoDB Manual*. MongoDB. 2025. URL: https://www.mongodb.com/docs/manual/core/wiredtiger/ (posećeno 1. 2. 2026.).